

Beyond Kalman Filters: Deep Learning-Based Filters for Improved Object Tracking

Paper Overview

Authors: Momir Adžemović, Predrag Tadić, Andrija Petrović, Mladen Nikolić

Main Goal: Replace traditional Kalman Filters with deep learning-based filters for better object tracking, especially with non-linear motion

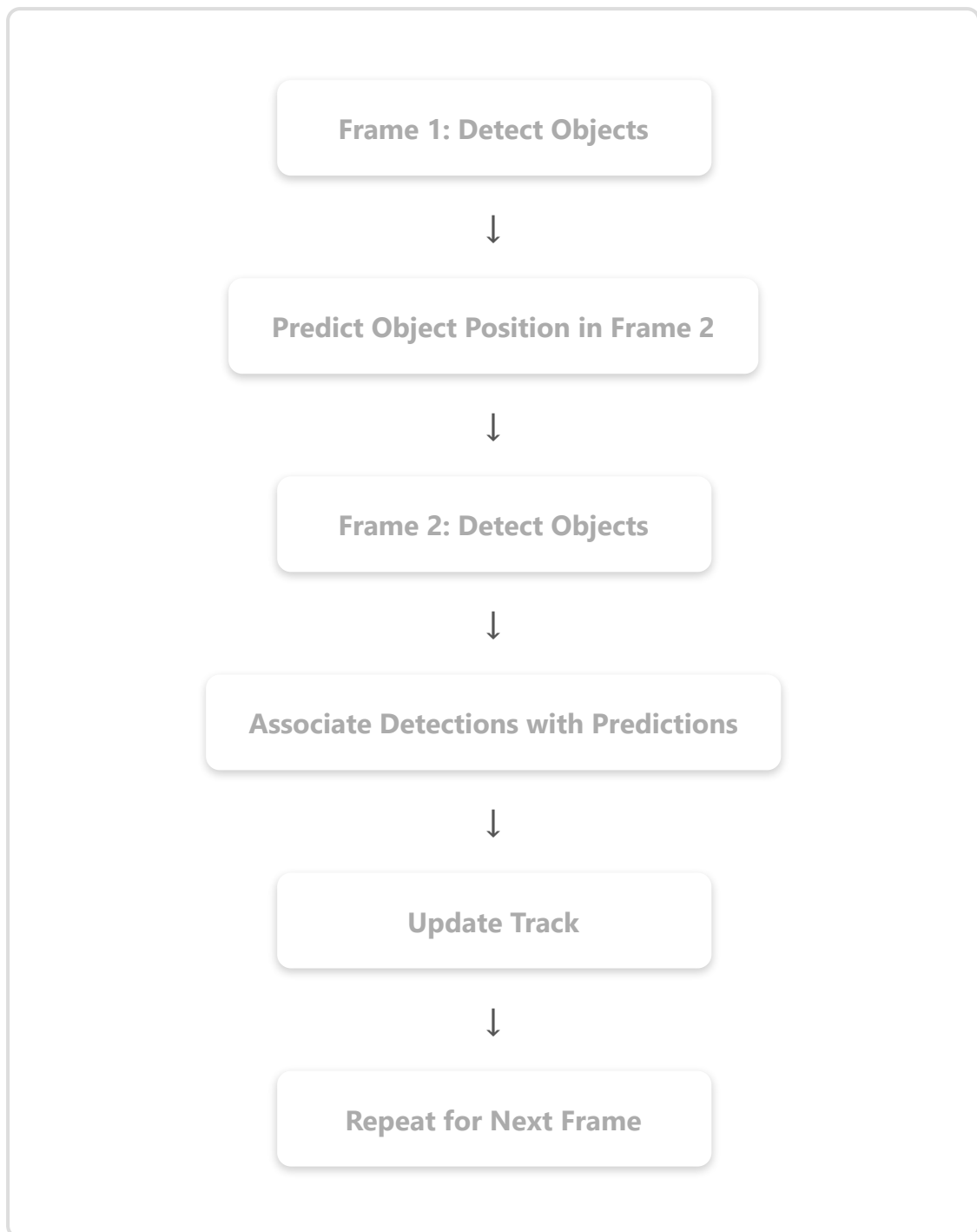
Introduction & Problem Statement

What is Object Tracking?

Following objects across video frames to understand their movements and behaviors in real-time. Used in:

-  Autonomous vehicles
-  Robotics
-  Surveillance systems

Tracking-by-Detection Paradigm



⚠ The Problem with Kalman Filter (KF)

- **Good for linear motion:** Works well when objects move in straight lines
- **Fails for non-linear motion:** Can't handle curves, sudden turns, or complex movements
- **Requires domain expertise:** Needs manual tuning of many parameters
- **Can't learn from data:** Fixed mathematical model

2 Main Contributions

✨ Contribution 1: Bayesian Filter

Uses a **trainable motion model** to predict where objects will be, then combines this with detector measurements using Bayes' rule

Benefit: Reduces need for manual hyperparameter tuning

✨ Contribution 2: End-to-End Filter

Learns to predict positions AND correct detector errors automatically

Benefit: No domain expertise needed, works with any detector

✨ Contribution 3: Motion Model Architectures

Three advanced neural network architectures:

- **RNN-based:** Recurrent Neural Networks for sequences
- **Neural ODEs:** Continuous-time models using differential equations
- **Conditional Neural Processes (CNP):** Flexible models for uncertain data

✨ Contribution 4: MoveSORT Tracker

New tracking system with improved association method that considers:

- Overlap between bounding boxes (IoU)
- Position differences
- Scale differences

3 Technical Background

3.1 Neural Ordinary Differential Equations (NODE)

What are Neural ODEs?

Instead of discrete steps (like traditional neural networks), NODEs model **continuous change over time** using differential equations

Traditional Neural Network vs Neural ODE

Aspect	Traditional Network	Neural ODE
Depth	Discrete layers (Layer 1, 2, 3...)	Continuous depth (infinite layers)
Time Steps	Fixed time points	Any time point
Memory	Stores all intermediate steps	More memory efficient
Noise Robustness	Moderate	High

Key Equation:

$$dh(t)/dt = f(h(t), t, \theta)$$

Where:

- $h(t)$ = hidden state at time t
- f = neural network function
- θ = trainable parameters

✅ Advantages of NODE

- Robust to noisy data
- Handles missing observations well
- Works at any time resolution
- Memory efficient

❌ Disadvantages of NODE

- Slower computation
- More complex to implement
- Requires ODE solver

3.2 Conditional Neural Processes (CNP)

What are CNPs?

Models that learn to represent patterns in data by conditioning on observed points to predict unobserved points. Like having a smart interpolator!

CNP Architecture Flow

Step 1: Encode

Each observed point (time, position) → latent representation

Step 2: Aggregate

Combine all representations → global context

Step 3: Decode

Global context + query time → predicted position

4 Proposed Methodologies

4.1 Motion Model Architectures

Complete Architecture Comparison

Architecture	Type	Key Feature	Speed	Accuracy
AR-RNN	Baseline	Simple recurrent model	Fast	Good
RNN-ODE	Hybrid	GRU + Neural ODE	Slow	Very Good
RNN-CNP	Neural Process	GRU aggregation	Medium	Excellent
ACNP	Neural Process	Attention mechanism	Medium	Excellent
NODEFilter	End-to-End	NODE + GRU filtering	Slowest	Best
RNNFilter	End-to-End	GRU + GRU filtering	Medium	Best

4.2 Bayesian Filtering Method

Bayesian Filter Workflow

Input: Past Observations
($x_1, x_2, \dots, x_n$ at times $t_1, t_2, \dots, t_n$)



Motion Model Predicts
Prior Distribution: $N(\hat{\mu}, \Sigma)$



New Detection x_{n+1}
Measurement Likelihood: $N(x_{n+1}, R)$



Apply Bayes' Rule
Combine Prior + Measurement



Posterior Estimation: $N(\tilde{\mu}, \tilde{\Sigma})$
(Final predicted position)

Bayes' Rule for Filtering:

Posterior Covariance:

$$\tilde{\Sigma} = (\hat{\Sigma}^{-1} + R^{-1})^{-1}$$

Posterior Mean:

$$\mu^{\sim} = \Sigma^{\sim}(\Sigma^{\wedge^{-1}}\mu^{\wedge} + R^{-1}x)$$

Where:

- $\mu^{\wedge}, \Sigma^{\wedge}$ = Predicted mean and covariance (prior)
- x, R = Detection and measurement noise
- $\mu^{\sim}, \Sigma^{\sim}$ = Final estimate (posterior)

4.3 End-to-End Filtering Method

Key Innovation

Instead of manually setting measurement noise parameters, the network **learns to filter noise automatically** during training!

End-to-End Filter Workflow

Input: Past Observations + Current Detection



Stage 1: Predict (using ODE or GRU)
Get Prior Estimate



Stage 2: Update (using GRU)
Combine Prior + Detection
Learn to Filter Noise



Output: Refined Position
(No manual noise tuning needed!)

NODEFilter Architecture Details

Initialize: $\tilde{z}_0$ = zero vector (empty trajectory)

Predict Step: $\hat{z}_{i+1} = \text{ODESolver}(\tilde{z}_i, t_i, t_{i+1})$
Extrapolate latent state forward in time

Update Step: $\tilde{z}_{i+1} = \text{GRU}(x_{i+1}, \hat{z}_{i+1})$
Incorporate new measurement

Output: $\tilde{\mu}, \tilde{\Sigma} = \text{MLP}(\tilde{z}_{i+1})$
Project to position space

4.4 MoveSORT: The Complete Tracker

SORT (Simple Online Real-time Tracker) Baseline

Standard method that uses Kalman Filter + IoU (Intersection over Union) for matching

MoveSORT = SORT + Two Improvements

1. **Better Filter:** Replace KF with deep learning filter
2. **Better Association:** New hybrid cost function

Hybrid Association Method

Traditional SORT:

$\text{Cost} = -\text{IoU}(\text{prediction}, \text{detection})$

MoveSORT:

$\text{Cost} = -\text{IoU}(\text{prediction}, \text{detection}) + \lambda \times \text{L1_distance}(\text{prediction}, \text{detection})$

Where $\lambda = 5$ (balance parameter)

L1_distance = Manhattan distance considering position AND scale

💡 Why the Hybrid Method?

Problem: IoU alone doesn't distinguish between objects at different positions/scales

Solution: Add position and scale information to make better matches during occlusions

Result: Fewer ID switches (objects losing their identity)

5 Experimental Evaluation

5.1 Datasets Used



DanceTrack



SportsMOT

- **Content:** Dancers
- **Challenge:** Non-linear motion, occlusions
- **Videos:** 100 (40 train, 25 val, 35 test)
- **Why:** Perfect for non-linear motion testing

- **Content:** Sports (basketball, volleyball, football)
- **Challenge:** Fast, variable-speed motion
- **Videos:** 240 (45 train, 45 val, 150 test)
- **Why:** Complex movement patterns

MOT17 & MOT20

- **Content:** Pedestrians
- **Challenge:** Linear motion (easier)
- **Videos:** 14 (MOT17), 8 (MOT20)
- **Why:** Standard benchmark (but not ideal for this work)

LaSOT

- **Content:** 70+ categories, single objects
- **Challenge:** Diverse motion, occlusions
- **Videos:** 1,400+ scenes
- **Why:** Test filter robustness to noise

5.2 Evaluation Metrics

Metric	Full Name	What It Measures
HOTA	Higher Order Tracking Accuracy	Overall tracking quality (detection + association)
DetA	Detection Accuracy	How well objects are detected
AssA	Association Accuracy	How well tracks are maintained over time
MOTA	Multi-Object Tracking Accuracy	Overall accuracy (sensitive to detector)
IDF1	ID F1 Score	Quality of ID assignments
IDSW	ID Switches	How often objects lose their identity (lower is better)

5.3 Key Results - DanceTrack

Performance Comparison on DanceTrack Test Set

Method	HOTA (%)	MOTA (%)	IDF1 (%)	Improvement vs SORT
SORT Baseline	47.9	91.8	50.8	-
SORT-Tuned	51.2	91.5	49.7	+3.3%
MoveSORT-KF	53.3	91.7	51.2	+5.4%
MoveSORT-RNN-CNP	55.9	91.8	56.5	+8.0%
MoveSORT-NODEFilter	56.1	91.8	56.0	+8.2%
OC_SORT	55.1	89.4	54.9	-



Key Achievements on DanceTrack:

- MoveSORT-NODEFilter achieved **56.1% HOTA** (best result)
- Outperformed traditional SORT by **8.2%**
- Beat transformer-based MotionTrack by **3.2%**
- Surpassed OC_SORT by **1.0%**

5.4 Key Results - SportsMOT

Performance on SportsMOT Test Set

Method	HOTA (%)	MOTA (%)	IDF1 (%)
ByteTrack	64.1	95.9	71.4
OC_SORT	73.7	96.5	74.0
MoveSORT-RNNFilter	74.6	96.7	76.9

SportsMOT Achievements:

- MoveSORT-RNNFilter: **74.6% HOTA** (best motion-based tracker)
- Outperformed OC_SORT by **0.9% HOTA** and **2.9% IDF1**
- **Ranked #1 on MOTA leaderboard** at time of submission

5.5 Robustness Analysis (LaSOT Dataset)

Test 1: Robustness to Gaussian Noise

Setup: Added increasing levels of Gaussian noise to ground truth bounding boxes

Filter	No Noise	$\sigma = 0.05$	$\sigma = 0.15$	$\sigma = 0.30$
Kalman Filter	79%	69%	48%	28%
AR-RNN (naive)	85%	75%	42%	18%
RNN-CNP	86%	77%	62%	42%
NODEFilter	87%	82%	71%	56%

Test 2: Robustness to Missing Detections

Setup: Randomly removed detections at different rates (0% to 80%)

Filter	0% Missing	20% Missing	50% Missing	80% Missing
Kalman Filter	79%	68%	42%	18%
RNN-CNP	86%	80%	68%	48%
NODEFilter	87%	83%	74%	58%

🔑 Key Insights from Robustness Tests

- End-to-end filters (NODEFilter, RNNFilter) are **most robust** to noise
- All deep learning filters significantly outperform Kalman Filter under noise
- Training with augmentations (simulated noise + missing data) is crucial
- Neural ODEs maintain accuracy even with 50%+ missing detections

5.6 Occlusion Handling Analysis

What is an Occlusion?

When an object is temporarily hidden (e.g., person walks behind a pole).
The tracker must find it again when it reappears.

Occlusion Performance on DanceTrack

Filter	Precision	Recall	F1-Score
Kalman Filter	64.4%	80.4%	71.5%
RNN-CNP	63.6%	83.8%	72.3%
NODEFilter	64.5%	84.1%	73.0%

Improvement: NODEFilter finds 3.7% more occluded objects than KF while maintaining precision!

5.7 Computational Efficiency

Speed Comparison (Frames Per Second)

Method	Predict FPS	Update FPS	Overall FPS	Speed Rating
Kalman Filter	26,147	11,493	7,984	⚡ ⚡ ⚡ Fastest
AR-RNN	1,462	16,637	1,397	⚡ ⚡ Fast
RNN-CNP	868	21,492	834	⚡ ⚡ Fast
RNNFilter	85	1,289	80	⚡ Moderate
NODEFilter	53	1,099	52	⚡ Moderate

⚖️ Speed vs Accuracy Trade-off

- **Need maximum speed?** Use Kalman Filter (but accept lower accuracy)
- **Good balance?** Use RNN-CNP (8x slower than KF, but much better accuracy)
- **Maximum accuracy?** Use NODEFilter or RNNFilter (slower but best results)
- **Real-time tracking?** RNN-CNP achieves 62.5 FPS with YOLOv8-M detector

6 Implementation Details

6.1 Training Setup

Training Configuration

- **Batch Size:** 256
- **Learning Rate:** 0.001 with step scheduler
- **Optimizer:** AdamW with weight decay
- **Loss Function:** Gaussian Negative Log-Likelihood
- **GPU:** Single NVIDIA RTX 3070 (8GB VRAM)

6.2 Data Augmentation

Augmentation Techniques

Augmentation	Purpose	Implementation
Gaussian Noise	Simulate detector errors	$\sigma = 0.05, 0.10, \text{ or } 0.25$ (random)
Remove Points	Simulate missing detections	20% probability per point
History Shortening	Handle varying trajectory lengths	5% probability, keep min 2 points

Why Augmentations Matter

Without augmentations, AR-RNN accuracy drops from **76%** to **44%** when noise is added. With augmentations, it stays at **68%**!

6.3 Feature Engineering

Input Feature Transformations

Raw Input: Bounding Box (top, left, width, height)



Choose Transformation



Method	Formula	Best For
Scaled First-Order Difference	$Y_i = (X_{i+1} - X_i) / (t_{i+1} - t_i)$	Linear motion (MOT17, MOT20)
Relative to Last	$Y_i = X_n - X_i$	Non-linear motion (DanceTrack, SportsMOT)

Why Transform Coordinates?

- **Translation Invariance:** Model doesn't need to learn about absolute positions
- **Faster Training:** 7% accuracy improvement + faster convergence
- **Noise Robustness:** Relative features more resistant to detector errors
- **Standardization:** Additional 0.1% accuracy gain

7 Practical Recommendations

Which Filter Should You Use?

Scenario	Recommended Filter	Why
Linear Motion + Speed Critical	Kalman Filter	Fast and sufficient for simple motion
Non-linear Motion + Need Speed	RNN-CNP	Best balance: 834 FPS, excellent accuracy
Complex Motion + High Noise	RNNFilter	Robust filtering, moderate speed (80 FPS)
Maximum Accuracy Required	NODEFilter	Best accuracy, handles severe occlusions
Resource Constrained	AR-RNN	Lightweight, better than KF for non-linear

When to Use Each Dataset for Training

✓ DanceTrack /

SportsMOT

- Non-linear motion patterns
- Frequent occlusions
- Similar object appearances
- Fast movements

Best for: Training general-purpose motion models

⚠ MOT17 / MOT20

- Mostly linear motion
- Pedestrian-specific
- Not ideal for testing deep learning filters

Best for: Benchmarking against traditional methods

8

Limitations & Future Work

Current Limitations

- **No appearance features:** Only uses motion (could be improved with visual features)
- **No camera motion compensation:** Could boost performance on moving camera scenes
- **Computational cost:** NODEFilter slower than KF (trade-off for accuracy)
- **Hyperparameter tuning:** Some parameters still dataset-specific (e.g., T_{lost})

Future Directions

- Incorporate **appearance features** (visual similarity)
- Add **camera motion compensation**
- Explore **Neural Process-based end-to-end filters**
- Optimize **inference speed** (ONNX, TensorRT)
- Test on **more diverse datasets**

Key Takeaways

Main Lessons

1. Deep Learning > Traditional Methods for Non-linear Motion

- All proposed filters outperform Kalman Filter on DanceTrack and SportsMOT
- Up to **8.2% HOTA improvement** on DanceTrack
- Especially effective during occlusions

2. End-to-End Learning Reduces Engineering

- NODEFilter and RNNFilter require minimal hyperparameter tuning
- Can adapt to different detectors without retraining
- Learn to filter noise automatically

3. Robustness Comes from Training

- Augmentations (noise + missing data) crucial for robustness
- Without augmentations: 76% → 44% accuracy under noise
- With augmentations: 76% → 68% accuracy under noise

4. Speed-Accuracy Trade-off Exists

- KF fastest (7,984 FPS) but less accurate
- RNN-CNP good balance (834 FPS, high accuracy)
- NODEFilter most accurate but slower (52 FPS)

5. Hybrid Association Helps

- IoU + L1 distance better than IoU alone
- 2.1% HOTA improvement just from better matching

- Reduces ID switches significantly

10 Glossary of Terms

Multi-Object Tracking (MOT)

Following multiple objects across video frames, maintaining their identities over time

Tracking-by-Detection

Detect objects in each frame, then connect detections across frames to form tracks

Kalman Filter (KF)

Traditional algorithm that uses linear equations to predict object positions. Good for straight-line motion.

IoU (Intersection over Union)

Overlap between two bounding boxes. $\text{IoU} = (\text{Area of Overlap}) / (\text{Area of Union})$. Range: 0 (no overlap) to 1 (perfect match)

ID Switch

When a tracker incorrectly changes an object's identity (e.g., Person #1 becomes Person #2)

Occlusion

When an object is temporarily hidden from view

Prior / Posterior

Prior: Prediction before seeing new data

Posterior: Updated estimate after incorporating new data

Bounding Box

Rectangle around object, defined by (x, y, width, height) or (top, left, width, height)

Latent Representation

Hidden internal representation learned by neural network that captures essential information

GRU (Gated Recurrent Unit)

Type of recurrent neural network that remembers past information for sequential data



Summary Flowchart

Complete MoveSORT Pipeline

Video Frame at time t



Object Detector (YOLOX)
Finds all objects



For each existing track:
Filter predicts position at time t



Calculate Cost Matrix
(IoU + L1 distance)



Hungarian Algorithm
Match detections to tracks



Update matched tracks with detections
Create new tracks for unmatched detections
Delete tracks with no match for T_{lost} frames



Output: Tracked objects with IDs



Final Verdict

This paper demonstrates that **deep learning-based filters can effectively replace Kalman Filters** for object tracking, especially in scenarios with **non-linear motion** and **frequent occlusions**.

The proposed methods are **more accurate, more robust**, and require **less manual tuning** than traditional approaches.



Full Citation:

Adžemović, M., Tadić, P., Petrović, A., & Nikolić, M. (2025). Beyond Kalman filters: deep learning-based filters for improved object tracking. Machine Vision and Applications, 36:20.